SOAP Delivery Integration

Assignment 3 | Project: Sweet Scoop

Team ID	10	Team members	Prerna (20251651072); Akanksha Updahyay (20251651011); Annu Kumari (20251651024, Team Leader)

1. Context and runtime boundary

The Order Service creates a delivery through the Delivery Service REST API. The Delivery Service builds a SOAP 1.1 `createShipment` envelope and sends it with HTTP POST, `Content-Type: text/xml; charset=utf-8`, and a `SOAPAction` header. The runnable project currently points `DELIVERY_PARTNER_URL` at the local gateway SOAP stub (`http://localhost:3000/v1/soap/delivery` by default). The stub returns a sample shipment identifier and tracking number; it is not a live carrier and does not validate partner credentials or transmit to a real external service.

The root `partner.wsdl` is the design-time partner contract loaded by the Delivery Service. Its external HTTPS example endpoint is illustrative only. The local app serves this WSDL from `GET /v1/soap/delivery`; `POST /v1/soap/delivery` returns the local stub response. A production integration must replace the URL, provision credentials through a secret manager, confirm the partner's actual contract, and validate response/fault handling before sending customer data.

2. SOAP operation contract

Operation	Request	Response	Fault
`createShipment`	Order ID, customer name, address, item list, delivery type	Shipment ID, status, tracking number, estimated time, partner reference	`ShipmentFault`
`getTrackingStatus`	Shipment ID	Current status and event list	`ShipmentFault`
`cancelShipment`	Shipment ID and optional reason	Shipment ID, cancellation result, message	`ShipmentFault`

Authentication note: `AuthHeader` is declared in the WSDL. `soap-request.xml` contains a conspicuous placeholder; the local stub does not require or verify it. Never put a real key in the example XML or assignment archive.

3. HTTP SOAP binding

The running service accepts SOAP XML using HTTP POST. The exact local request is created by `buildCreateShipmentEnvelope()` in `backend/services/delivery/index.js`, and the request uses the WSDL's `createShipment` SOAPAction. It can be inspected locally at the gateway endpoint after starting the app. The files `soap-request.xml`, `soap-response.xml`, and `soap-fault.xml` are sanitized contract examples, not packet captures from the local stub or a real partner.

```
POST /v1/soap/delivery HTTP/1.1
Host: localhost:3000
Content-Type: text/xml; charset=utf-8
SOAPAction: "http://partner.sweetscoop.example.com/delivery/createShipment"
```

4. Discovery record

Property	Value
Contract	WSDL 1.1, document/literal SOAP
Local discovery URL	`http://localhost:3000/v1/soap/delivery`
Design-time external endpoint	HTTPS example URL in `partner.wsdl`; not live
Service / port	`DeliveryPartnerService` / `DeliveryPartnerPort`
Operations	`createShipment`, `getTrackingStatus`, `cancelShipment`
Runtime consumer	Delivery Service

5. Fault mapping and resilience

The adapter recognizes SOAP fault envelopes and maps known partner codes to sanitized Problem Details; it retains the partner code but does not expose the partner's raw message or XML.

Partner code	HTTP	Problem type	Handling
`INVALID_ADDRESS`	400	`/errors/bad-request`	Address syntax/required fields are invalid.
`ADDRESS_NOT_SERVICEABLE`	422	`/errors/validation-failed`	Address is valid but outside the partner's service area.
`RATE_LIMITED`	429	`/errors/rate-limited`	Includes `Retry-After: 5`.
`PARTNER_UNAVAILABLE`	502	`/errors/bad-gateway`	Required partner operation failed.
Unknown code	502	`/errors/bad-gateway`	Fail closed as an upstream integration error.

The mapping is covered with mocked partner faults in the Delivery route tests; the local SOAP stub currently only returns a success envelope. When the Order saga receives a known address-validation 422, it compensates completed payment/inventory work before returning a sanitized 422 to its caller.

The Delivery Service wraps its outbound request in a circuit breaker and the shared retry helper. The helper uses a five-second timeout and only retries this unsafe POST when an idempotency key is supplied; the partner call currently does not pass one, so ambiguous unsafe calls are not automatically replayed. Repeated failures open the breaker. The current fallback logs a failed non-fault call and may create a local delivery record without a partner shipment reference; this is an explicit demo limitation, not a confirmed external shipment.

6. Artifacts

File	Description
`partner.wsdl`	SOAP contract and HTTPS example endpoint.
`soap-request.xml`	Sanitized request example with placeholder credential.
`soap-response.xml`	Representative success envelope.
`soap-fault.xml`	Representative partner fault.

Runtime boundary reminder
This document distinguishes design-time examples from actual local runtime behavior.